

Architectures matérielles

1. Des transistors aux portes logiques

1.1. Le transistor : un interrupteur piloté

Un transistor possède trois bornes et peut être vu, dans ce contexte, comme un interrupteur commandé.

- Une **borne de commande** reçoit en entrée un signal logique : une tension basse correspond au 0, une tension haute correspond au 1 (c'est ainsi que la logique binaire se traduit en électronique).
- Les deux autres bornes forment un chemin électrique distinct, que ce signal de commande peut ouvrir ou fermer.
 - Si la commande active le transistor, le courant passe : l'interrupteur est fermé.
 - Sinon, le courant est bloqué : l'interrupteur est ouvert.

Selon le type de transistor et le montage utilisé, la sortie obtenue peut être directe ou inversée. Mais l'idée essentielle est qu'**un signal logique permet de commander le passage du courant**.

En associant des transistors de manière appropriée, on construit des **portes logiques**, briques élémentaires de tout circuit numérique. Un processeur moderne en contient plusieurs *milliards*. Ces portes logiques manipulent des valeurs booléennes — 0 ou 1, vrai ou faux — et réalisent des opérations définies par la *logique booléenne*.

1.2. Rappels de logique booléenne

Inventée par le mathématicien George Boole, c'est l'algèbre (les « règles de calcul ») du *vrai* et du *faux* :

- elle ne manipule que deux valeurs, 0 (faux) et 1 (vrai) ;
- et comporte trois opérations fondamentales à partir desquelles toute expression logique peut être construite.

Opération	Notation mathématique	Python	Sens
NON (NOT)	$\bar{a}$	<code>not a</code>	Inverse la valeur (1 $\rightarrow$ 0 ; 0 $\rightarrow$ 1)
ET (AND)	$a \cdot b$	<code>a and b</code>	1 si et seulement si les deux entrées sont à 1
OU (OR)	$a + b$	<code>a or b</code>	1 si au moins une entrée est à 1

Remarque : sens de ces « formules » ? ▼

- L'opération $\cdot$ est bien la multiplication : quand a et b prennent pour valeur 0 ou 1, $a \times b = a \cdot b$ vaut 1 si et seulement si a **et** b valent 1. Toutes les autres combinaisons donnent 0 pour résultat. Cela correspond bien à l'opération logique **ET**.
- L'opération $+$ est à comprendre de façon un peu particulière : si $a = 1$ et $b = 1$, alors *dans ce contexte* $a + b$ vaudra 1 (et non 2, au contraire de l'addition ordinaire). Alors, pour que le résultat $a + b$ vaille 1, il faut et il suffit que a **ou** b vaille 1 : c'est bien l'opération logique **OU**.

Enfin, les **lois de De Morgan** sont deux relations bien utiles :

$$\overline{a \cdot b} = \bar{a} + \bar{b} \qquad \overline{a + b} = \bar{a} \cdot \bar{b}$$

Avec les opérations logiques :

$NON(A \text{ ET } B) = (NON \text{ } A) \text{ OU } (NON \text{ } B)$ $NON(A \text{ OU } B) = (NON \text{ } A) \text{ ET } (NON \text{ } B)$.

Elles permettent de transformer toute combinaison de ET et de OU en son équivalent exprimé avec l'opération inverse — ce qui est fondamental en électronique (voir plus loin les portes logiques universelles).

Remarque : minuscules et majuscules dans les expressions ▼

Par convention, on utilise des minuscules (*a, b, s...*) pour désigner des variables booléennes (les valeurs 0 ou 1 manipulées dans les formules mathématiques), et des majuscules (*A, B, S...*) pour désigner les signaux électriques aux entrées et sorties des portes logiques dans les schémas de circuits. Mais cette distinction n'est pas toujours respectée...

1.3. Les portes logiques

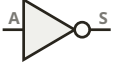
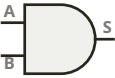
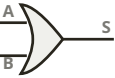
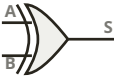
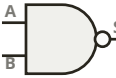
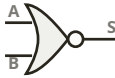
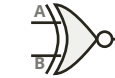
Une **porte logique** est un circuit élémentaire qui réalise une opération booléenne. Les différentes portes sont listées ci-dessous.

Dans la symbologie **européenne**, chaque porte est représentée par un *rectangle* avec un symbole intérieur assez explicite : c'est lui qui indique la nature de la porte. Un petit cercle tangent au rectangle en sortie indique une négation.

Symbole	Nom de la porte	Table de vérité	Description															
	NON ou NOT	<table><tr><td>A</td><td>S</td></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	S	0	1	1	0	Inverse le signal. S vaut 1 si A vaut 0, et _vice versa_. Expression : $s = \bar{a}$									
A	S																	
0	1																	
1	0																	
	ET ou AND	<table><tr><td>A</td><td>B</td><td>S</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	S	0	0	0	0	1	0	1	0	0	1	1	1	S vaut 1 <i>seulement si</i> A et B valent 1 simultanément. Expression : $s = a \cdot b$
A	B	S																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
	OU ou OR	<table><tr><td>A</td><td>B</td><td>S</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	S	0	0	0	0	1	1	1	0	1	1	1	1	S vaut 1 si A ou B (ou les deux) valent 1. Expression : $s = a + b$
A	B	S																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
	OU exclusif ou XOR	<table><tr><td>A</td><td>B</td><td>S</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	S	0	0	0	0	1	1	1	0	1	1	1	0	S vaut 1 si A et B sont <i>différents</i> (l'un et pas l'autre). Expression : $s = a \oplus b = a \cdot \bar{b} + \bar{a} \cdot b$
A	B	S																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
	NON-ET ou NAND	<table><tr><td>A</td><td>B</td><td>S</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	S	0	0	1	0	1	1	1	0	1	1	1	0	Porte universelle ! Inverse de ET . S vaut 0 <i>seulement si</i> A et B valent 1. Expression : $s = \overline{a \cdot b}$
A	B	S																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
	NON-OU ou NOR	<table><tr><td>A</td><td>B</td><td>S</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	S	0	0	1	0	1	0	1	0	0	1	1	0	Porte universelle ! Inverse de OU . S vaut 1 <i>seulement si</i> A et B valent 0. Expression : $s = \overline{a + b}$
A	B	S																
0	0	1																
0	1	0																
1	0	0																
1	1	0																

Remarque : symboles européens vs américains ▼

La table précédente montre les symboles **européens**. Les **symboles américains** ont des formes différentes, qui identifient chaque porte. Ces symboles, antérieurs à leurs équivalents européens, sont très répandus dans les ressources en ligne (on les trouve même dans au moins un sujet de bac... français !).

NON	ET	OU	OU exclusif	NON-ET	NON-OU	NON-OU exclusif
						

1.4. Porte logique universelle

✓ Portes universelles : NAND et NOR

La porte **NAND** seule (ou **NOR** seule) suffit à construire *toutes* les autres portes — et donc tout circuit numérique.

On dit que ces portes sont **universelles** ou **fonctionnellement complètes**.

✏ Exemple : construire avec NAND et NOR ▼

Recréons logiquement les portes **NOT**, **AND** et **OR** à l'aide de la seule porte **NAND**.

- **NOT** : relier les deux entrées ensemble $\rightarrow \text{NAND}(A, A) = \text{NOT}(A)$.
- **AND** : un **NAND** suivi d'un **NOT** (lui-même fait avec un **NAND**).
- **OR** : on l'obtient depuis ce **AND** et la formule de De Morgan indiquée en premier ci-dessus. En appliquant **NOT** des deux côtés, on trouve $A \text{ OR } B = \text{NOT}(\text{NOT}(A) \text{ AND } \text{NOT}(B))$. On a pu exprimer **NOT** et **AND** à partir de **NAND** : c'est gagné !

On peut faire de même avec la porte **NOR**.

- **NOT** : relier les deux entrées ensemble $\rightarrow \text{NOR}(A, A) = \text{NOT}(A)$.
- **OR** : un **NOR** suivi d'un **NOT** (lui-même fait avec un **NOR**).
- **AND** : maintenant qu'on a le **OR**, il suffit d'utiliser l'autre loi de De Morgan.

1.5. Circuits combinatoires

Un **circuit combinatoire** est un assemblage de portes logiques dont la sortie ne dépend que des entrées *actuelles* (pas d'état mémorisé). Pour analyser un circuit, on construit sa **table de vérité** en appliquant chaque porte successivement.

✏ Exemple : demi-additionneur 1 bit ▼

Pour additionner deux bits a et b :

- **Somme** $s = a \oplus b$ (porte XOR)
- **Retenue** $r = a \cdot b$ (porte ET)

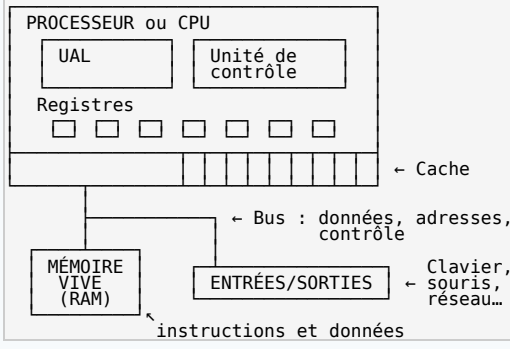
a	b	s	r
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

En combinant des demi-additionneurs pour former des additionneurs complets, puis en les chaînant, on obtient un **additionneur n bits**, capable d'additionner des nombres de n bits en propageant correctement les éventuelles retenues : c'est un élément central de l'UAL (voir ci-après).

2. Architecture Von Neumann

Proposée en 1945 par John von Neumann, c'est le modèle de référence de la quasi-totalité des ordinateurs actuels. Il repose sur plusieurs **composants** reliés par un **bus** :

2.1. Organisation schématique

Schéma	Composant	Rôle
	UAL	L'Unité Arithmétique et Logique effectue les calculs arithmétiques et les opérations logiques
	UC	L'Unité de Contrôle décode les instructions et pilote leur séquençement
	Registres	Mémoires de petite taille internes au CPU, ultra-rapides
	RAM	Mémoire de travail qui stocke à la fois les instructions et les données. Elle est volatile (perdue hors tension)

Idee fondamentale : programme et données partagent la *même* mémoire et le *même* bus — c'est le principe du **programme enregistré** (*stored program*).

Hiérarchie des mémoires (de la plus rapide à la plus lente) : **Registres** > **Cache** > **RAM** > **Stockage de masse** (SSD / disque dur).

 **Goulot d'étranglement de Von Neumann**

Instructions et données empruntent le *même* bus : le CPU ne peut pas charger une instruction et lire une donnée *simultanément*. C'est le principal défaut de cette architecture.

2.2. Registres importants

Un registre est une toute petite mémoire ultra-rapide, située à l'intérieur même du processeur, qui stocke temporairement une valeur le temps d'un calcul ou d'une opération. Trois registres jouent un rôle central :

- **PC (Program Counter, compteur de programme)** : contient l'adresse mémoire de la prochaine instruction à exécuter. Après chaque instruction, il s'incrémente automatiquement pour passer à la suivante — c'est lui qui « sait où on en est » dans le programme.
- **IR (Instruction Register, registre d'instruction)** : contient l'instruction en cours de traitement. Une fois lue en mémoire, l'instruction y est copiée le temps que l'unité de contrôle la décode et l'exécute.
- **Accumulateur** : registre de travail de l'UAL. C'est là que sont déposés les opérandes (valeurs à traiter) et que l'UAL écrit le résultat d'un calcul avant de le renvoyer en mémoire ou de l'utiliser dans la suite du programme.

 **Remarque : position de l'accumulateur au sein du CPU ▼**

Historiquement, l'accumulateur des premiers processeurs (type 8080 ou Z80) était le registre de travail *intégré* à l'UAL, qu'elle seule pouvait utiliser, câblé directement à ses entrées / sorties.

C'est encore vrai pour certains microcontrôleurs. Mais sur les processeurs modernes, l'accumulateur n'est plus un registre spécifique : tous les registres généraux peuvent jouer ce rôle.

2.3. Cycle d'exécution

Le processeur répète indéfiniment le même enchaînement de trois phases, appelé cycle *Fetch – Decode – Execute* (en français : Récupérer – Décoder – Exécuter, ou Lecture – Décodage – Exécution) :

1. **Fetch** (récupérer) : le processeur lit en mémoire l'instruction dont l'adresse est dans le PC, la copie dans l'IR, puis incrémente le PC.
2. **Decode** (décoder) : l'unité de contrôle analyse l'instruction contenue dans l'IR pour déterminer quelle opération effectuer et quelles données utiliser.
3. **Execute** (exécuter) : l'UAL effectue l'opération (calcul, lecture/écriture en mémoire, saut...) et écrit le résultat.

Puis le cycle recommence immédiatement au *Fetch* — des *milliards* de fois par seconde sur un processeur moderne.

2.4. Fonctionnement global

Un programme est, de nos jours, une suite d'instructions écrites dans un **langage de haut niveau** (C, Java, Python...), conçu pour être lisible par un humain. Ces instructions doivent être traduites en **langage machine** — une suite d'opérations élémentaires binaires que le processeur (ou CPU, pour **C**entral **P**rocessing **U**nity) exécute grâce à ses circuits implémentés matériellement (on dit souvent « câblés en dur » mais c'est un abus de langage).

Cette traduction est assurée par un **compilateur** (pour C, C++...) ou un **interpréteur** accompagné d'une **machine virtuelle** (pour Python). Python fait ainsi une étape intermédiaire : le code source est d'abord converti en *bytecode* (un code *intermédiaire*), puis ce *bytecode* est exécuté par la machine virtuelle Python. Au final, comme pour tout programme, ce sont bien des instructions machine natives qui sont exécutées par le processeur.

Le résultat final est dans tous les cas le même : une suite d'opérations élémentaires que le CPU traite une par une dans son cycle *Fetch – Decode – Execute*.

Il existe donc deux niveaux de langage bien distincts :

- Le langage de haut niveau (voire « très haut niveau » pour des langages tels que Python), expressif et lisible, dans lequel le développeur écrit son programme.
- Le langage machine (suite de bits), très difficilement lisible pour un humain (c'est pourtant ainsi que les tout premiers programmes étaient écrits !), mais seul langage que le processeur comprend réellement.

Chaque opération élémentaire est identifiée par un code binaire unique, appelé **opcode**. C'est cet *opcode* — suivi des données ou adresses nécessaires — que le processeur va lire en RAM à l'adresse indiquée par le **registre PC** lors de la phase *Fetch*.

L'ensemble des **opcodes** disponibles sur un processeur est son **jeu d'instructions**, qui varie selon les architectures (x86 pour les PC, ARM pour les smartphones et les puces Apple récentes).

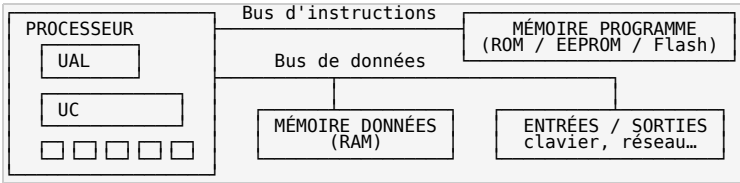
3. Architecture Harvard et microcontrôleurs

L'architecture de **Harvard** a été mise en œuvre dès 1944, à l'université du même nom.

3.1. Organisation schématique

Contrairement à l'architecture proposée par Von Neumann, celle de Harvard **sépare physiquement** les mémoires d'instructions et de données, avec des bus distincts pour les relier au processeur : cela élimine le goulot d'étranglement de l'architecture concurrente.

En effet, le CPU peut *simultanément* lire la prochaine instruction (*via* le bus d'instructions) et lire/écrire une donnée (*via* le bus de données) — là où Von Neumann voit son bus devoir alterner (d'où ledit goulot d'étranglement).



3.2. Harvard vs Von Neumann

	Von Neumann	Harvard
Mémoire	Unique (instructions + données)	Deux mémoires séparées
Bus	Unique	Deux bus distincts
Parallélisme	Limité	Instructions et données en parallèle
Usage typique	PC, serveurs	Microcontrôleurs, DSP

Remarque : et de nos jours ? ▼

L'architecture de Harvard est toujours utilisée dans les microcontrôleurs ([Arduino](#), [PIC](#)...) et les processeurs de signal numérique (ou *DSP*).

Pour la culture : les processeurs modernes (x86, ARM) utilisent *en apparence* une architecture Von Neumann : instructions et données cohabitent dans la même mémoire principale (la RAM). Mais, pour gagner en vitesse, ils sont dirigés en interne vers deux caches L1 différents : l'un dédié aux instructions, l'autre aux données. Ces deux caches fonctionnent exactement selon le principe Harvard : deux chemins d'accès parallèles, sans conflit. On parle d'*architecture de Harvard modifiée* : Von Neumann côté mémoire principale, Harvard côté cache.

Note : on parle de cache L1 car il existe souvent différents niveaux de mémoire cache au sein d'un processeur.

3.3. Microcontrôleurs et informatique embarquée

Un **microcontrôleur** ([Arduino](#), [Raspberry Pi Pico](#), [STM32](#)...) s'apparente à un **SoC** (*System on a Chip*) simplifié conçu pour les systèmes embarqués (des systèmes autonomes — encombrement limité, fonctionnement sur batteries, devant fonctionner en conditions difficiles... — ayant des besoins informatiques mais des ressources très contraintes pour les satisfaire).

Il intègre sur une même puce CPU, mémoire (souvent Harvard ou Harvard modifiée), et des entrées/sorties directement connectables à des capteurs et actionneurs.

- **Capteur :** convertit une grandeur physique (température, distance...) en signal électrique lisible par le CPU. C'est une **entrée**.
- **Actionneur :** convertit une commande du CPU en action physique (moteur, LED, buzzer...). C'est une **sortie**.

Monde physique → [Capteur] → CPU → [Actionneur] → Monde physique

4. L'assembleur

4.1. Présentation

Un processeur n'exécute que du *langage machine* : une suite de bits que les circuits interprètent directement. Ce langage est difficilement utilisable par un humain : une instruction se présente sous la forme d'un opcode binaire comme `10110000 01100001`.

Le *langage assembleur* (souvent abrégé en *assembleur* tout court) en est la transcription lisible : chaque instruction machine y est représentée par un **mnémotique**, c'est-à-dire une abréviation évocatrice — `MOV` pour déplacer une valeur, `ADD` pour additionner, `JMP` pour « sauter » à une adresse.

L'opération qui consiste à convertir du code assembleur en langage machine binaire s'appelle l'**assemblage**, et l'outil qui l'effectue est **UN assembleur** (attention à la confusion !).

Remarque : illisible pour un humain ? et pourtant... ▼

Les tout premiers programmes informatiques — saisis sur cartes perforées dans les années 1940-1950 — étaient directement écrits en langage machine : chaque trou perforé à la bonne position encodait un bit, et le programmeur devait connaître par cœur les *opcodes* binaires du processeur.

L'assembleur a précisément été inventé pour épargner aux programmeurs cette fastidieuse manipulation de bits.

L'assembleur est un langage de **très bas niveau** : il ne dispose ni de boucles (`for` , `while`), ni de fonctions au sens habituel, ni de types de données — chaque ligne correspond généralement à **une instruction machine**. Il est en cela radicalement différent des langages de haut niveau comme C ou Python.

4.2. Note culturelle

Il est rare de nos jours de programmer directement en assembleur : la plupart du code « bas niveau » est écrit en C ou en C++ (Rust prend de l'ampleur). Cependant, l'assembleur reste utile — voire indispensable ponctuellement — dans certains contextes très proches du matériel.

- **Microcode des CPU** : dans certains processeurs, des instructions complexes sont décomposées en micro-opérations internes plus élémentaires. Cette traduction est pilotée par un microcode propre au processeur. Ce n'est pas de l'assembleur au sens habituel, mais cela montre qu'il existe encore des niveaux de programmation situés « sous » le langage machine visible par le programmeur. Il y a donc littéralement du code **dans** le CPU !
- **Firmwares** : les logiciels embarqués dans des composants matériels (carte réseau, disque dur, BIOS/UEFI, microcontrôleurs...) pilotent le matériel au plus bas niveau. Ils sont le plus souvent écrits en C/C++, mais peuvent contenir de petites portions d'assembleur lorsque le contrôle précis du matériel ou du démarrage l'exige.
- **Optimisations dans les pilotes GPU** : les *shaders* sont de petits programmes exécutés massivement en parallèle par les milliers de cœurs d'un GPU, pour calculer l'éclairage, les ombres ou les effets visuels d'une scène 3D. Les pilotes graphiques (NVIDIA, AMD...) contiennent des portions d'assembleur GPU finement optimisées pour tirer le maximum de performances dans des jeux ou applications spécifiques.
- **Systèmes embarqués critiques** : dans l'automobile, l'aérospatiale ou le médical, certains logiciels doivent respecter des contraintes fortes (exécution en temps réel, sûreté, consommation...). Ils sont généralement écrits dans des langages comme C, mais certaines portions très sensibles peuvent être codées ou vérifiées au niveau assembleur pour maîtriser précisément ce que fait le processeur.

4.3. Exemple : instructions de base (jeu RISC-V)

Inutile de retenir cela par cœur ! Ce thème a été abordé dans au moins un sujet de bac, et les informations utiles y ont été intégralement exposées.

Instruction	Effet
<code>addi rA, rB, v</code>	$rA \leftarrow rB + v$ (v : valeur immédiate)
<code>add rA, rB, rC</code>	$rA \leftarrow rB + rC$
<code>sub rA, rB, rC</code>	$rA \leftarrow rB - rC$
<code>beq rA, rB, label</code>	Saute à <code>label</code> si $rA == rB$
<code>bne rA, rB, label</code>	Saute à <code>label</code> si $rA \neq rB$
<code>bgt rA, rB, label</code>	Saute à <code>label</code> si $rA > rB$
<code>ble rA, rB, label</code>	Saute à <code>label</code> si $rA \leq rB$

Une **étiquette** (`LABEL :`) identifie une position dans le code vers laquelle un branchement peut sauter.

4.4. Lire un programme assembleur

Ce qui est attendu à l'examen

Il ne *devrait* pas être demandé de programmer en assembleur, mais savoir *lire* une courte séquence et *dérouler son exécution* en suivant les valeurs des registres instruction par instruction l'a déjà été (cf. [26-NSIJ2G11](#)).

Exemple : maximum de deux registres ▼

On veut mettre dans `r3` le maximum de `r1` et `r2`.

```
bgt r1, r2, PLUS_GRAND    # si r1 > r2, sauter à PLUS_GRAND
add r3, r2, r0             # sinon r3 ← r2  (r0 vaut toujours 0 en RISC-V)
beq r1, r1, FIN           # astuce : r1 est toujours égal à r1 ⇒ saut inconditionnel
PLUS_GRAND:
add r3, r1, r0            # r3 ← r1
FIN:
```

Remarque : ce qu'on a ici est en réalité une simplification de l'assembleur RISC-V.

Exemple : l'échange de registres (un classique !) ▼

Pour échanger `r1` et `r2`, on utilise `r3` comme registre temporaire.

```
add r3, r1, r0    # r3 ← r1  (sauvegarde)
add r1, r2, r0    # r1 ← r2
add r2, r3, r0    # r2 ← ancienne valeur de r1
```

Cette situation se rencontre à l'identique dans *tous* les langages de programmation quels que soient leurs « niveaux ». Néanmoins, Python facilite l'échange des valeurs de deux variables `a` et `b` en permettant d'écrire simplement `a, b = b, a`.